

# AI应用开发实战：RAG知识库助手实战-附带前后端和完整可运行服务



## RAG 知识库问答系统 - 从零开始的完整指南

基于 Milvus 向量数据库 + LangChain + DashScope 的本地知识库问答解决方案

 直接可以运行的代码已经整理好了，需要的话找老师拿

小D智库助手

知识库设置

点击或拖拽文件至此

支持 PDF, DOCX, TXT, CSV

集合名称

agent\_rag

立即解析入库

智能问答

当前集合: agent\_rag

你好

你好！有什么问题我可以帮助你吗？

参考依据 (5)

二当家小D是谁？

内容检索+数据入库+云端milvus+精准匹配+RAG检索增强+模型接入

二当家小D是小滴课堂的核心讲师之一，现任小D课堂后端技术总监。他拥有8年开发与架构经验，曾就职于阿里巴巴、荔枝FM等知名互联网公司，主导过多个日均亿级流量的系统，并独立架构过千万用户级别的第三方开放平台。他精通Java语言，擅长分布式系统、数据处理、支付架构等技术领域，是小滴课堂的技术核心人物，致力于为学员提供贴近实战的IT课程内容。

参考依据 (5)

#1

相似度: 1141754.2%

\*\*发展历程\*\* 2018-2019年，小滴课堂专注课程录制，在自有平台、51CTO学院、腾讯课堂等学习平台积累学习人数50万+；2020年，拓展IT岗位阶段技术课程，开展电商大平台品牌加盟店，积累用户100万+；2021年，推出大课训练营+小班辅导课，进行多品类课程拓展。 --- ## 二当家小D介绍 二当家小D是小滴课堂的核心讲师之一。就职于阿里巴巴和荔枝FM等知名互联网公司。小D老师主导过多个日均数亿流量的系统，独立架构过千万用户级别的第三方开放平台。他拥有8年开发架构经验，精通Java语言，擅长分布式系统、数据处理、支付架构等技术领域，目前担任小D课堂后端技术总监一职。作为小滴课堂的技术核心人物，二当家小D凭借其丰富的大厂工作经验和扎实的技术功底，为学员带来贴近实战的课程内容，帮助众多IT从业者提升技术能力。

#2

相似度: 1141754.2%

\*\*发展历程\*\* 2018-2019年，小滴课堂专注课程录制，在自有平台、51CTO学院、腾讯课堂等学习平台积累学习人数50万+；2020年，拓展IT岗位阶段技术课程，开展电商大平台品牌加盟店，积累用户100万+；

输入您的问题，按 Enter 发送...

按 Enter 发送，Shift + Enter 换行

## 1. 什么是 RAG？

### 1.1 RAG 的定义

**RAG（Retrieval-Augmented Generation，检索增强生成）** 是一种将**信息检索**与**大语言模型（LLM）生成**相结合的技术。

用一个简单的比喻来理解：

- 想象你在参加一场开卷考试。RAG 就像是：
- 📖 **你的参考书**（知识库）
  - 🔍 **快速翻书找答案的能力**（检索系统）
  - 🧠 **理解并组织答案的大脑**（大语言模型）

### 1.2 为什么需要 RAG？



传统大语言模型存在几个问题：

| 问题   | 说明         | RAG 如何解决   |
|------|------------|------------|
| 知识过时 | 模型训练后不再更新  | 可以随时更新知识库  |
| 幻觉问题 | 模型可能"编造"答案 | 基于真实文档生成回答 |
| 领域限制 | 缺乏专业/私有知识  | 可导入任意专业文档  |
| 不可追溯 | 无法知道答案来源   | 可以引用原始文档   |

1.3 RAG 的工作流程

代码块



## 2. 什么是 Milvus?

### 2.1 向量数据库简介

在理解 Milvus 之前，我们需要先了解什么是**向量**：

**向量（Vector）** 是一组数字，用来表示文本、图片等数据的"语义特征"。

例如：

代码块

```
1  "猫"   → [0.23, -0.45, 0.78, ..., 0.12]  （1536维向量）
2  "狗"   → [0.21, -0.42, 0.75, ..., 0.15]  （相似度高！）
3  "汽车" → [0.89, 0.23, -0.56, ..., 0.67]  （相似度低）
```

**向量数据库** 就是专门用来存储和快速搜索这些向量的数据库。

### 2.2 Milvus 是什么？

**Milvus** 是一个开源的向量数据库，专门为 AI 应用设计：

- 🚀 **高性能**：支持亿级向量的毫秒级搜索
- 📊 **可扩展**：支持分布式部署
- 🔧 **易集成**：提供 Python、Java、Go 等多种 SDK
- 💾 **持久化**：数据可靠存储，支持备份恢复

### 2.3 Milvus 的核心概念

| 概念         | 类比   | 说明         |
|------------|------|------------|
| Collection | 数据库表 | 存储向量数据的容器  |
| Entity     | 表中的行 | 一条向量数据记录   |
| Field      | 表中的列 | 向量字段、标量字段等 |
| Index      | 索引   | 加速搜索的数据结构  |

### 2.4 本项目中的 Milvus 配置

代码块

```
1  # 连接配置
2  MILVUS_HOST = "localhost"
3  MILVUS_PORT = "19530"
4
5  # 集合 Schema
6  fields = [
7      FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),
8      FieldSchema(name="name", dtype=DataType.VARCHAR, max_length=255),
9      FieldSchema(name="text", dtype=DataType.VARCHAR, max_length=5000),
10     FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536)
11 ]
```

### 3. 项目架构概览

#### 3.1 整体架构图

本项目的架构可以分为两大流程：

##### 文档索引流程（Indexing Pipeline）

代码块

```
1  本地文档 → DocumentLoader → TextSplitter → Embedding Model → Milvus
2      ①           ②           ③           ④           ⑤
```

##### 检索问答流程（Retrieval Pipeline）

代码块

```
1  用户提问 → Query Embedding → Vector Search → Top-K Results → LLM → 答案
2      ⑥           ⑦           ⑧           ⑨           ⑩      ⑪
```

#### 3.2 技术栈

| 组件     | 技术选型                        | 用途          |
|--------|-----------------------------|-------------|
| 向量数据库  | Milvus                      | 存储和检索向量     |
| 应用框架   | LangChain                   | 构建 RAG 流水线  |
| 嵌入模型   | DashScope text-embedding-v1 | 文本向量化       |
| 大语言模型  | Qwen-Plus                   | 生成回答        |
| Web 框架 | Flask                       | 提供 REST API |
| 文档处理   | PyPDF、Docx2txt 等            | 解析各类文档      |

### 4. 核心概念详解





## 4.1 文本切分 (Text Splitting)

为什么需要切分文本?

- 嵌入模型有长度限制:** 大多数模型最多处理 512-8192 个 token
- 检索精度:** 较短的文本块更容易精确匹配
- 上下文控制:** LLM 的上下文窗口有限

本项目使用 `RecursiveCharacterTextSplitter`:

代码块

```
1 self.text_splitter = RecursiveCharacterTextSplitter(  
2     chunk_size=500,           # 每块最大 500 字符  
3     chunk_overlap=50,        # 相邻块重叠 50 字符  
4     separators=["\n\n", "\n", ". ", "! ", "? ", "; ", ", ", " ", ""]  
5 )
```

## 4.2 向量嵌入 (Embedding)

嵌入是将文本转换为数值向量的过程:

代码块

```
1 "Milvus是一个向量数据库"  
2     ↓  
3     Embedding Model  
4     ↓  
5 [0.023, -0.156, 0.892, ..., 0.445] # 1536维向量
```

本项目使用阿里云 DashScope 的 `text-embedding-v1` 模型, 输出 1536 维向量。

## 4.3 相似度搜索 (Similarity Search)

向量相似度通常使用以下方法计算:

| 方法    | 公式                         | 特点     |
|-------|----------------------------|--------|
| L2 距离 | $\sqrt{\sum(a_i - b_i)^2}$ | 值越小越相似 |
| 余弦相似度 | $\cos(\theta)$             | 值越大越相似 |
| 内积    | $\sum(a_i \times b_i)$     | 值越大越相似 |

本项目使用 **L2 距离**（欧氏距离）。

## 5. 项目文件结构

代码块

```
1  project/
2  |—— vector_db_manager.py      # 核心：向量数据库管理器
3  |—— document_loader.py        # 文档加载器（支持多格式）
4  |—— vector_retriever.py       # 向量检索器 + 问答
5  |—— query_system.py          # 独立查询系统（命令行）
6  |—— upload_document.py        # 独立上传脚本（命令行）
7  |—— api_integration.py        # Flask API 蓝图
8  |—— server.py                 # Flask 服务器入口
9  |—— .env                      # 环境变量配置
```

## 文件职责说明

| 文件                                   | 主要类/函数                | 职责                        |
|--------------------------------------|-----------------------|---------------------------|
| <a href="#">vector_db_manager.py</a> | VectorDatabaseManager | 连接 Milvus、文档处理、向量存储       |
| <a href="#">document_loader.py</a>   | DocumentLoader        | 加载 PDF/TXT/DOCX/CSV/Excel |
| <a href="#">vector_retriever.py</a>  | VectorRetriever       | 封装搜索逻辑、生成回答               |
| <a href="#">query_system.py</a>      | SimpleQuerySystem     | 独立的 RAG 查询实现              |
| <a href="#">api_integration.py</a>   | Flask Blueprint       | REST API 端点定义             |

## 6. 关键代码讲解

### 6.1 向量数据库管理器 (vector\_db\_manager.py)

这是项目的核心模块，负责整个 RAG 的"写入"流程。

#### 初始化连接

代码块

```
1  class VectorDatabaseManager:
2      def __init__(self, milvus_host, milvus_port, ...):
3          # 1. 配置参数
4          self.milvus_host = milvus_host or os.getenv("MILVUS_HOST", "127.0.0.1")
5          self.milvus_port = str(milvus_port or os.getenv("MILVUS_PORT", "19530"))
6
7          # 2. 初始化嵌入模型
```

```

8         self._init_embeddings()
9
10        # 3. 初始化文本切分器
11        self.text_splitter = RecursiveCharacterTextSplitter(
12            chunk_size=500,
13            chunk_overlap=50,
14            separators=["\n\n", "\n", "。 ", "! ", "? ", "; ", ", ", " ", ""]
15        )
16
17        # 4. 连接到 Milvus
18        self._connect_to_milvus()

```

## 文档处理流水线

代码块

```

1  def process_file(self, file_path: str, collection_name: str = None) -> bool:
2      """处理单个文件：加载 → 切分 → 存储"""
3
4      # 步骤1：加载文档
5      documents = self.load_document(file_path)
6      # 例如：PDF 文件会被解析为多个 Document 对象
7
8      # 步骤2：切分文档
9      split_docs = self.split_documents(documents)
10     # 将长文档切分为多个小块
11
12     # 步骤3：存入向量数据库
13     self.add_documents_to_db(split_docs, collection_name)
14     # 自动调用嵌入模型，生成向量并存储
15
16     return True

```

## 向量存储逻辑

代码块

```

1  def add_documents_to_db(self, documents: List[Document], collection_name: str = None):
2      """将文档添加到 Milvus"""
3
4      target_collection = collection_name or self.collection_name
5      collection_exists = utility.has_collection(target_collection)
6
7      if collection_exists:
8          # 集合已存在：加载并追加数据
9          self.vectorstore = Milvus(
10              embedding_function=self.embeddings,
11              collection_name=target_collection,
12              connection_args={"host": self.milvus_host, "port": self.milvus_port}
13          )
14          self.vectorstore.add_documents(documents) # 追加
15      else:
16          # 集合不存在：创建新集合并插入
17          self.vectorstore = Milvus.from_documents(
18              documents=documents,
19              embedding=self.embeddings,
20              collection_name=target_collection,
21              connection_args={"host": self.milvus_host, "port": self.milvus_port}
22          )

```



## 6.2 向量检索器 (vector\_retriever.py)

这个模块负责 RAG 的"读取"流程。

### 相似度搜索

代码块

```
1 def search_similar_content(self, query: str, collection_name: str, k: int = None):
2     """搜索相似内容"""
3
4     # 执行向量搜索 (内部会自动将 query 转为向量)
5     search_results = self.db_manager.search(
6         query=query,
7         k=k,
8         collection_name=collection_name
9     )
10
11     # 过滤低相似度结果
12     results = []
13     for doc, score in search_results:
14         if score >= self.similarity_threshold: # 默认 0.5
15             results.append((doc, score))
16
17     return results
```

### 问答生成

代码块

```
1 def answer_question(self, question: str, collection_name: str, k: int = 5):
2     """回答问题 - RAG 的核心逻辑"""
3
4     # 步骤1: 检索相关文档
5     relevant_docs = self.search_similar_content(
6         query=question,
7         collection_name=collection_name,
8         k=k
9     )
10
11     # 步骤2: 构建上下文
12     context_parts = []
13     for i, (doc, score) in enumerate(relevant_docs):
14         context_parts.append(f"参考资料{i+1}: {doc.page_content}")
15     context = "\n\n".join(context_parts)
16
17     # 步骤3: 调用 LLM 生成回答
18     answer = self._generate_answer_with_llm(question, context)
19
20     return AnswerResult(
21         answer=answer,
22         source_documents=source_documents,
23         scores=scores
24     )
```

### LLM 调用

代码块

```
1 def _generate_answer_with_llm(self, question: str, context: str) -> str:
2     """使用 LLM 生成回答"""
```

```

3
4     client = OpenAI(
5         api_key=os.environ.get("DASHSCOPE_API_KEY"),
6         base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
7     )
8
9     system_prompt = """你是一个智能助手。请基于提供的【参考资料】回答用户的问题。
10    如果参考资料为空或与问题无关，请利用你的通用知识进行回答..."""
11
12    response = client.chat.completions.create(
13        model="qwen-plus",
14        messages=[
15            {"role": "system", "content": system_prompt},
16            {"role": "user", "content": f"问题: {question}\n\n【参考资料】: \n{context}"}
17        ]
18    )
19
20    return response.choices[0].message.content

```

## 6.3 API 接口 (api\_integration.py)

提供 REST API 供外部调用。

### 上传文档端点

代码块

```

1  @vector_bp.route('/upload_document', methods=['POST'])
2  def upload_document():
3      """POST /api/vector/upload_document"""
4
5      data = request.get_json()
6      file_path = data['file_path']
7      collection_name = data['collection_name']
8
9      # 调用核心处理逻辑
10     success = vector_manager.process_file(file_path, collection_name)
11
12     if success:
13         return jsonify({
14             'success': True,
15             'message': f'文档处理成功: {file_path}',
16             'database_info': vector_manager.get_database_info(collection_name)
17         })

```

### 查询端点

代码块

```

1  @vector_bp.route('/query', methods=['POST'])
2  def query_documents():
3      """POST /api/vector/query"""
4
5      data = request.get_json()
6      question = data['question']
7      collection_name = data['collection_name']
8      k = data.get('k', 5)
9
10     # 执行 RAG 问答
11     result = vector_retriever.answer_question(

```

```
12         question,
13         k=k,
14         collection_name=collection_name
15     )
16
17     return jsonify({
18         'success': True,
19         'question': question,
20         'answer': result.answer,
21         'confidence': result.confidence,
22         'sources': [...] # 来源文档信息
23     })
```

## 7. 快速上手指南

### 7.1 环境准备

#### 4. 启动 Milvus（使用 Docker）

代码块

```
1 # 创建 docker-compose.yml 并启动
2 docker-compose up -d
```

#### 5. 安装 Python 依赖

代码块

```
1 pip install langchain langchain-community langchain-milvus
2 pip install pymilvus dashscope openai
3 pip install flask flask-cors python-dotenv
4 pip install pypdf docx2txt pandas openpyxl
```

#### 6. 配置环境变量

创建 `.env` 文件：

代码块

```
1 MILVUS_HOST=localhost
2 MILVUS_PORT=19530
3 COLLECTION_NAME=agent_rag
4 DASHSCOPE_API_KEY=your_api_key_here
5 EMBEDDING_MODEL=text-embedding-v1
6 LLM_MODEL=qwen-plus
```

### 7.2 命令行使用

上传文档：

代码块

```
1 # 修改 upload_document.py 中的 FILE_PATH
2 python upload_document.py
```

查询问答：

代码块

```
1  # 修改 query_system.py 中的 QUESTION
2  python query_system.py
```

7.3 API 使用

启动服务：

代码块

```
1  python server.py
```

上传文档：

代码块

```
1  curl -X POST http://localhost:5000/api/vector/upload_document \
2    -H "Content-Type: application/json" \
3    -d '{"file_path": "/path/to/doc.pdf", "collection_name": "my_docs"}'
```

提问：

代码块

```
1  curl -X POST http://localhost:5000/api/vector/query \
2    -H "Content-Type: application/json" \
3    -d '{"question": "什么是RAG?", "collection_name": "my_docs", "k": 5}'
```

8. API 接口说明

| 端点                           | 方法   | 描述            |
|------------------------------|------|---------------|
| /api/vector/upload_document  | POST | 上传并处理文档（路径方式） |
| /api/vector/upload_file      | POST | 上传文件流         |
| /api/vector/query            | POST | RAG 问答        |
| /api/vector/search           | POST | 纯相似度搜索        |
| /api/vector/collection_info  | GET  | 获取集合信息        |
| /api/vector/clear_collection | POST | 清空集合          |